In [1]:
```
"Import pandas as pd"
```

Out[1]:
```
'Import pandas as pd'
```

In [9]:
```
import pandas as pd

sbux = pd.read_csv("BUS303_Signature_Term_Project_Starbucks_Beverage_Menu.cs

sbux.isnull().sum()
```

Out[9]:
```
Product_Name           1
Size                   1
Milk                   1
Whip                   1
Serving_Size_ml        1
Calories               1
Total_Fat_g            1
Saturated_Fat_g        2
Trans_Fat_g            1
Cholesterol_mg         1
Sodium_mg              1
Carbohydrates_g        1
Dietary_Fiber_g        1
Sugar_g                1
Protein_g              1
Vitamin_A_%DV          1
Vitamin_C_%DV          1
Calcium_%DV            1
Iron_%DV               1
Caffiene_milligrams    1
dtype: int64
```

In [10]:
```
sbux.head()
```

Out[10]:

	Product_Name	Size	Milk	Whip	Serving_Size_ml	Calories	Total_Fat_g	Satu
0	Brewed Coffee - Dark Roast	Short	No Milk	No Whip	236.0	3.0	0.1	
1	Brewed Coffee - Dark Roast	Tall	No Milk	No Whip	354.0	4.0	0.1	
2	Brewed Coffee - Dark Roast	Grande	No Milk	No Whip	473.0	5.0	0.1	
3	Brewed Coffee - Dark Roast	Venti¬Æ	No Milk	No Whip	591.0	5.0	0.1	
4	Brewed Coffee Traveler - Dark Roast	1 - 236 mL serving	No Milk	No Whip	236.0	3.0	0.1	

In [12]:
```
sbux2 = sbux.dropna(how='all')
```

In [13]:
```
sbux2.isnull().sum()
```

Out[13]:
```
Product_Name           0
Size                   0
Milk                   0
Whip                   0
Serving_Size_ml        0
Calories               0
Total_Fat_g            0
Saturated_Fat_g        1
Trans_Fat_g            0
Cholesterol_mg         0
Sodium_mg              0
Carbohydrates_g        0
Dietary_Fiber_g        0
Sugar_g                0
Protein_g              0
Vitamin_A_%DV          0
Vitamin_C_%DV          0
Calcium_%DV            0
Iron_%DV               0
Caffiene_milligrams    0
dtype: int64
```

In [14]:
```
sbux2.dtypes
```

Out[14]:
```
Product_Name           object
Size                   object
Milk                   object
Whip                   object
Serving_Size_ml       float64
Calories              float64
Total_Fat_g           float64
Saturated_Fat_g       float64
Trans_Fat_g           float64
Cholesterol_mg        float64
Sodium_mg             float64
Carbohydrates_g       float64
Dietary_Fiber_g       float64
Sugar_g                object
Protein_g             float64
Vitamin_A_%DV         float64
Vitamin_C_%DV         float64
Calcium_%DV           float64
Iron_%DV              float64
Caffiene_milligrams   float64
dtype: object
```

In [15]:
```
sbux2.iloc[1, 13]
```

Out[15]:
```
'zero'
```

In [22]:
```
sbux2.dtypes
```

Out[22]:
```
Product_Name            object
Size                    object
Milk                    object
Whip                    object
Serving_Size_ml        float64
Calories               float64
Total_Fat_g            float64
Saturated_Fat_g        float64
Trans_Fat_g            float64
Cholesterol_mg         float64
Sodium_mg              float64
Carbohydrates_g        float64
Dietary_Fiber_g        float64
Sugar_g                float64
Protein_g              float64
Vitamin_A_%DV          float64
Vitamin_C_%DV          float64
Calcium_%DV            float64
Iron_%DV               float64
Caffiene_milligrams    float64
dtype: object
```

In [23]:
```
sbux2.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 1204 entries, 0 to 1204
Data columns (total 20 columns):
 #   Column               Non-Null Count  Dtype  
---  ------               --------------  -----  
 0   Product_Name         1204 non-null   object 
 1   Size                 1204 non-null   object 
 2   Milk                 1204 non-null   object 
 3   Whip                 1204 non-null   object 
 4   Serving_Size_ml      1204 non-null   float64
 5   Calories             1204 non-null   float64
 6   Total_Fat_g          1204 non-null   float64
 7   Saturated_Fat_g      1203 non-null   float64
 8   Trans_Fat_g          1204 non-null   float64
 9   Cholesterol_mg       1204 non-null   float64
 10  Sodium_mg            1204 non-null   float64
 11  Carbohydrates_g      1204 non-null   float64
 12  Dietary_Fiber_g      1204 non-null   float64
 13  Sugar_g              1204 non-null   float64
 14  Protein_g            1204 non-null   float64
 15  Vitamin_A_%DV        1204 non-null   float64
 16  Vitamin_C_%DV        1204 non-null   float64
 17  Calcium_%DV          1204 non-null   float64
 18  Iron_%DV             1204 non-null   float64
 19  Caffiene_milligrams  1204 non-null   float64
dtypes: float64(16), object(4)
memory usage: 197.5+ KB
```

In [24]:
```
df2 = sbux2.dropna(subset=["Saturated_Fat_g"])
```

In [25]:
```
df2.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 1203 entries, 0 to 1204
Data columns (total 20 columns):
 #   Column               Non-Null Count  Dtype  
---  ------               --------------  -----  
 0   Product_Name         1203 non-null   object 
 1   Size                 1203 non-null   object 
 2   Milk                 1203 non-null   object 
 3   Whip                 1203 non-null   object 
 4   Serving_Size_ml      1203 non-null   float64
 5   Calories             1203 non-null   float64
 6   Total_Fat_g          1203 non-null   float64
 7   Saturated_Fat_g      1203 non-null   float64
 8   Trans_Fat_g          1203 non-null   float64
 9   Cholesterol_mg       1203 non-null   float64
 10  Sodium_mg            1203 non-null   float64
 11  Carbohydrates_g      1203 non-null   float64
 12  Dietary_Fiber_g      1203 non-null   float64
 13  Sugar_g              1203 non-null   float64
 14  Protein_g            1203 non-null   float64
 15  Vitamin_A_%DV        1203 non-null   float64
 16  Vitamin_C_%DV        1203 non-null   float64
 17  Calcium_%DV          1203 non-null   float64
 18  Iron_%DV             1203 non-null   float64
 19  Caffiene_milligrams  1203 non-null   float64
dtypes: float64(16), object(4)
memory usage: 197.4+ KB
```

In [26]:
```
sbux3 = df2.rename(columns={"Caffiene_milligrams": "Caffiene_mg"})
```

In [27]:
```
sbux3.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 1203 entries, 0 to 1204
Data columns (total 20 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   Product_Name     1203 non-null   object
 1   Size             1203 non-null   object
 2   Milk             1203 non-null   object
 3   Whip             1203 non-null   object
 4   Serving_Size_ml  1203 non-null   float64
 5   Calories         1203 non-null   float64
 6   Total_Fat_g      1203 non-null   float64
 7   Saturated_Fat_g  1203 non-null   float64
 8   Trans_Fat_g      1203 non-null   float64
 9   Cholesterol_mg   1203 non-null   float64
 10  Sodium_mg        1203 non-null   float64
 11  Carbohydrates_g  1203 non-null   float64
 12  Dietary_Fiber_g  1203 non-null   float64
 13  Sugar_g          1203 non-null   float64
 14  Protein_g        1203 non-null   float64
 15  Vitamin_A_%DV    1203 non-null   float64
 16  Vitamin_C_%DV    1203 non-null   float64
 17  Calcium_%DV      1203 non-null   float64
 18  Iron_%DV         1203 non-null   float64
 19  Caffiene_mg      1203 non-null   float64
dtypes: float64(16), object(4)
memory usage: 197.4+ KB
```

In [28]:
```
sbux3.to_csv("sbux4.csv", index=False)
```

In [29]:
```
sbux4 = pd.read_csv("sbux4.csv")
```

In [30]:
```
sbux4.head()
```

Out[30]:

	Product_Name	Size	Milk	Whip	Serving_Size_ml	Calories	Total_Fat_g	Satu
0	Brewed Coffee - Dark Roast	Short	No Milk	No Whip	236.0	3.0	0.1	
1	Brewed Coffee - Dark Roast	Tall	No Milk	No Whip	354.0	4.0	0.1	
2	Brewed Coffee - Dark Roast	Grande	No Milk	No Whip	473.0	5.0	0.1	
3	Brewed Coffee - Dark Roast	Venti¬Æ	No Milk	No Whip	591.0	5.0	0.1	
4	Brewed Coffee Traveler - Dark Roast	1 - 236 mL serving	No Milk	No Whip	236.0	3.0	0.1	

In [9]:
```
# Deliverable Two

import pandas as pd
```

```
dnkn = pd.read_csv("BUS_303_Dunkin_Donuts_Nutritional_Menu_Kaggle_2023.csv")

dnkn.head()
```

Out[9]:

	Category	Item	Calories	Total Fat (g)	Saturated Fat (g)	Trans Fat (g)	Cholesterol (mg)	Sodium (mg)	Carbs (g)
0	Cold Brew Coffee	Cold Brew Coffee, Large	5	0	0	0	0	20	0
1	Cold Brew Coffee	Cold Brew Coffee, Medium	5	0	0	0	0	15	0
2	Cold Brew Coffee	Cold Brew Coffee, Small	5	0	0	0	0	10	0
3	Cold Brew Coffee	Cold Brew Coffee with Cream, Large	120	12	6	0	35	65	2
4	Cold Brew Coffee	Cold Brew Coffee with Cream, Medium	90	9	4.5	0	25	45	1

In [10]:
```
dnkn.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 780 entries, 0 to 779
Data columns (total 13 columns):
 #   Column               Non-Null Count  Dtype 
---  ------               --------------  ----- 
 0   Category             780 non-null    object
 1   Item                 780 non-null    object
 2   Calories             780 non-null    object
 3   Total Fat (g)        780 non-null    object
 4   Saturated Fat (g)    780 non-null    object
 5   Trans Fat (g)        780 non-null    object
 6   Cholesterol (mg)     780 non-null    object
 7   Sodium (mg)          780 non-null    object
 8   Carbs (g)            780 non-null    object
 9   Fiber (g)            780 non-null    object
 10  Sugars (g)           780 non-null    object
 11  Protein (g)          780 non-null    object
 12  Weight Watcher Pnts  780 non-null    object
dtypes: object(13)
memory usage: 79.3+ KB
```

In [8]:
```
dnkn.isnull().sum()
```

Out[8]:
```
Category              0
Item                  0
Calories              0
Total Fat (g)         0
Saturated Fat (g)     0
Trans Fat (g)         0
Cholesterol (mg)      0
Sodium (mg)           0
Carbs (g)             0
Fiber (g)             0
Sugars (g)            0
Protein (g)           0
Weight Watcher Pnts   0
dtype: int64
```

In [12]:
```
dnkn.duplicated().sum()
```

Out[12]:
```
np.int64(1)
```

In [13]:
```
dnkn[dnkn.duplicated(keep=False)]
```

Out[13]:

	Category	Item	Calories	Total Fat (g)	Saturated Fat (g)	Trans Fat (g)	Cholesterol (mg)	Sodium (mg)	Car (
737	Sandwiches	Ham and Cheese Rollups	280	13	6	0	40	1050	
760	Sandwiches	Ham and Cheese Rollups	280	13	6	0	40	1050	

In [14]:
```
dnkn.nunique()
```

Out[14]:
```
Category                23
Item                   779
Calories                87
Total Fat (g)           50
Saturated Fat (g)       29
Trans Fat (g)            5
Cholesterol (mg)        37
Sodium (mg)            112
Carbs (g)              120
Fiber (g)                9
Sugars (g)             122
Protein (g)             32
Weight Watcher Pnts    362
dtype: int64
```

In [4]:
```
dnkn["Category"].value_counts()
```

Out[4]:
```
Category
Hot Coffee              88
Donuts                  87
Hot Latte               63
Iced Coffee             63
Iced Latte              63
Sandwiches              55
Frozen Coffee           49
Hot Macchiato           45
Hot Cappuccino          43
Iced Macchiato          42
Iced Cappuccino         42
Iced Tea                33
Hot Chocolate           28
Muffins                 18
Kolache                 16
Cold Brew Coffee        11
Coolatta                 9
Dunkin,Äô Refreshers     9
Soft Serve               8
Hot Americano            3
Iced Americano           3
Hash Browns              1
Dunkin,Äô Bowls          1
Name: count, dtype: int64
```

In [5]:
```
dnkn.describe(include='all')
```

Out[5]:

	Category	Item	Calories	Total Fat (g)	Saturated Fat (g)	Trans Fat (g)	Cholesterol (mg)	Sodium (mg)	Ca
count	780	780	780	780	780	780	780	780	
unique	23	779	87	50	29	5	37	112	
top	Hot Coffee	Ham and Cheese Rollups	170	0	0	0	0	180	
freq	88	2	38	242	269	755	222	24	

In [13]:
```
dnkn['Total Fat (g)'].unique()
```

Out[13]:
```
array(['0', '12', '9', '6', '4.5', '3', '10', '11', '28', '30', '22',
       '21', '23', '14', '15', '20', '19', '16', '18', '25', '31', '27',
       '17', '37', '40', '5', '8', '35', '26', '13', '7', '¬†', '4', '1',
       '0.5', '1.5', '2.5', '2', '3.5', '42', '33', '24', '36', '54',
       '29', '32', '34', '52', '41', '43'], dtype=object)
```

In [14]:
```
dnkn['Saturated Fat (g)'].unique()
```

Out[14]:
```
array(['0', '6', '4.5', '3', '3.5', '2.5', '1.5', '4', '12', '14', '9',
       '10', '11', '7', '5', '8', '13', '18', '22', '17', '15', '¬†',
       '0.5', '19', '21', '1', '2', '20', '16'], dtype=object)
```

In [15]:
```
dnkn['Trans Fat (g)'].unique()
```

Out[15]:
```
array(['0', '1.5', '1', '0.5', '¬†'], dtype=object)
```

In [16]:
```
dnkn['Cholesterol (mg)'].unique()
```

Out[16]:
```
array(['0', '35', '25', '20', '15', '10', '50', '30', '40', '130', '100',
       '70', '105', '80', '55', '<5', '¬†', '125', '45', '60', '185',
       '340', '180', '95', '65', '190', '360', '195', '265', '175', '220',
       '120', '200', '240', '205', '110', '115'], dtype=object)
```

In [17]:
```
dnkn['Sodium (mg)'].unique()
```

Out[17]:
```
array(['20', '15', '10', '65', '45', '30', '5', '60', '320', '240', '160',
       '300', '310', '470', '450', '420', '150', '340', '290', '400',
       '390', '440', '280', '135', '380', '480', '270', '430', '460',
       '360', '200', '370', '190', '330', '250', '170', '220', '180',
       '130', '230', '125', '100', '115', '75', '350', '260', '120', '¬†',
       '95', '70', '50', '25', '85', '55', '90', '140', '105', '110',
       '210', '510', '570', '630', '500', '750', '620', '80', '40', '35',
       '1020', '810', '1260', '1460', '600', '410', '840', '1060', '800',
       '820', '1170', '520', '540', '670', '640', '1200', '590', '1190',
       '1210', '1080', '1240', '1980', '1010', '650', '960', '1140',
       '1220', '1270', '900', '910', '1050', '980', '1500', '1120', '710',
       '550', '850', '490', '760', '970', '580', '1160', '700', '680'],
      dtype=object)
```

In [18]:
```
dnkn['Fiber (g)'].unique()
```

Out[18]:
```
array(['0', '1', '2', '3', '4', '¬†', '6', '5', '8'], dtype=object)
```

In [19]:
```
dnkn['Protein (g)'].unique()
```

Out[19]:
```
array(['1', '0', '2', '10', '7', '5', '4', '6', '3', '8', '11', '9', '¬†',
       '12', '16', '15', '14', '13', '27', '24', '37', '17', '19', '26',
       '21', '23', '18', '28', '33', '22', '25', '20'], dtype=object)
```

In [20]:
```
dnkn['Carbs (g)'].unique()
```

Out[20]:
```
array(['0', '2', '1', '37', '27', '18', '112', '84', '56', '172', '129',
       '86', '113', '85', '31', '44', '57', '47', '63', '39', '55', '48',
       '36', '58', '33', '41', '34', '50', '40', '21', '42', '61', '43',
       '32', '52', '38', '59', '35', '45', '30', '49', '24', '176', '135',
       '95', '175', '134', '94', '180', '139', '97', '184', '142', '99',
       '141', '138', '183', '117', '88', '177', '160', '120', '80', '¬†',
       '191', '147', '103', '195', '150', '105', '111', '83', '110', '13',
       '10', '7', '65', '64', '73', '70', '53', '71', '51', '87', '72',
       '66', '89', '82', '12', '9', '6', '26', '67', '5', '4', '46', '3',
       '28', '19', '17', '25', '23', '16', '15', '79', '78', '60', '11',
       '20', '54', '14', '69', '22', '75', '77', '29', '74', '68', '8',
       '92'], dtype=object)
```

In [21]:
```
dnkn1 = dnkn.copy()
```

In [80]:
```
food_items = [
    'Sandwiches',
    'Donuts',
    'Muffins',
    'Kolache',
    'Hash Browns',
    'Dunkin,Äô Bowls']
```

In [81]:
```
dnkn2 = dnkn1[~dnkn1['Category'].isin(food_items)].copy()
```

In [82]:
```
dnkn2.shape
```

Out[82]: (602, 13)

In [83]:
```
dnkn1['Category'].isin(food_items).sum()
```

Out[83]: np.int64(178)

In [84]:
```
dnkn2.duplicated().sum()
```

Out[84]: np.int64(0)

In [85]:
```
dnkn2['Cholesterol (mg)'] = dnkn2['Cholesterol (mg)'].replace('<5', '5')
```

In [86]:
```
for col in dnkn2.select_dtypes(include='object').columns:
    dnkn2[col] = (
        dnkn2[col]
        .str.replace('\xa0', '', regex=False)
        .str.replace('¬†', '', regex=False)
        .str.strip()
    )
```

In [87]:
```
for col in dnkn2.columns:
    if dnkn2[col].dtype == 'object':
        print('\n', col)
        print(dnkn2[col].unique()[:50])
```

```
 Category
['Cold Brew Coffee' 'Coolatta' 'Frozen Coffee' 'Hot Americano'
 'Hot Cappuccino' 'Hot Chocolate' 'Hot Coffee' 'Hot Latte' 'Hot Macchiato'
 'Iced Americano' 'Iced Cappuccino' 'Iced Coffee' 'Iced Latte'
 'Iced Macchiato' 'Iced Tea' 'Dunkin,Äô Refreshers' 'Soft Serve']

 Item
['Cold Brew Coffee, Large' 'Cold Brew Coffee, Medium'
 'Cold Brew Coffee, Small' 'Cold Brew Coffee with Cream, Large'
 'Cold Brew Coffee with Cream, Medium'
 'Cold Brew Coffee with Cream, Small'
 'Cold Brew Coffee with Cream and Sugar, Large'
 'Cold Brew Coffee with Cream and Sugar, Medium'
 'Cold Brew Coffee with Cream and Sugar, Small'
 'Nitro Cold Brew Coffee, Medium' 'Nitro Cold Brew Coffee, Small'
 'Blue Raspberry Coolatta, Large' 'Blue Raspberry Coolatta, Medium'
 'Blue Raspberry Coolatta, Small' 'Vanilla Bean Coolatta, Large'
 'Vanilla Bean Coolatta, Medium' 'Vanilla Bean Coolatta, Small'
 'Pineapple Coolatta, Large' 'Pineapple Coolatta, Medium'
 'Pineapple Coolatta, Small'
 'Caramel Swirl Frozen Coffee with Cream, Large'
 'Caramel Swirl Frozen Coffee with Cream, Medium'
 'Caramel Swirl Frozen Coffee with Cream, Small'
 'Caramel Swirl Frozen Coffee with Skim Milk, Large'
 'Caramel Swirl Frozen Coffee with Skim Milk, Medium'
 'Caramel Swirl Frozen Coffee with Skim Milk, Small'
 'Caramel Swirl Frozen Coffee with Whole Milk, Large'
 'Caramel Swirl Frozen Coffee with Whole Milk, Medium'
 'Caramel Swirl Frozen Coffee with Whole Milk, Small'
 'French Vanilla Swirl Frozen Coffee with Cream, Large'
 'French Vanilla Swirl Frozen Coffee with Cream, Medium'
 'French Vanilla Swirl Frozen Coffee with Cream, Small'
 'French Vanilla Swirl Frozen Coffee with Skim Milk, Large'
 'French Vanilla Swirl Frozen Coffee with Skim Milk, Medium'
 'French Vanilla Swirl Frozen Coffee with Skim Milk, Small'
 'French Vanilla Swirl Frozen Coffee with Whole Milk, Large'
 'French Vanilla Swirl Frozen Coffee with Whole Milk, Medium'
 'French Vanilla Swirl Frozen Coffee with Whole Milk, Small'
 'Frozen Coffee with Cream, Large' 'Frozen Coffee with Cream, Medium'
 'Frozen Coffee with Cream, Small' 'Frozen Coffee with Skim Milk, Large'
 'Frozen Coffee with Skim Milk, Medium'
 'Frozen Coffee with Skim Milk, Small'
 'Frozen Coffee with Whole Milk, Large'
 'Frozen Coffee with Whole Milk, Medium' 'Frozen Dunkaccino, Large'
 'Frozen Dunkaccino, Medium' 'Frozen Dunkaccino, Small'
 'Frozen Vanilla Chai/Vanilla Spice, Large']

 Calories
['5' '120' '90' '60' '260' '190' '130' '460' '350' '230' '790' '590' '390'
 '340' '1120' '860' '610' '830' '650' '470' '880' '680' '490' '1110' '640'
 '780' '500' '370' '250' '550' '410' '930' '710' '510' '400' '' '1160'
 '890' '630' '720' '480' '360' '240' '530' '10' '70' '45' '170' '110'
 '160']

 Total Fat (g)
['0' '12' '9' '6' '4.5' '3' '40' '31' '22' '5' '11' '8' '35' '26' '17'
```

```
 '13' '14' '10' '7' '' '4' '1' '0.5' '23' '21' '15' '16' '1.5' '2.5' '2'
 '18' '3.5' '20' '19' '27']

 Saturated Fat (g)
['0' '6' '4.5' '3' '3.5' '2.5' '1.5' '22' '17' '12' '7' '18' '14' '9' '15'
 '10' '5' '' '0.5' '4' '19' '8' '21' '13' '11' '1']

 Trans Fat (g)
['0' '1.5' '1' '0.5' '']

 Cholesterol (mg)
['0' '35' '25' '20' '15' '10' '130' '100' '70' '40' '30' '105' '80' '55'
 '5' '' '125' '50' '45' '60']

 Sodium (mg)
['20' '15' '10' '65' '45' '30' '5' '60' '320' '240' '160' '290' '220'
 '180' '130' '280' '150' '230' '125' '200' '100' '115' '75' '350' '260'
 '120' '' '480' '370' '250' '430' '330' '95' '70' '50' '25' '85' '55'
 '135' '90' '170' '140' '105' '110' '190' '210' '510' '470' '340' '570']

 Carbs (g)
['0' '2' '1' '37' '27' '18' '112' '84' '56' '172' '129' '86' '113' '85'
 '180' '139' '97' '184' '142' '99' '141' '138' '183' '57' '117' '88' '59'
 '177' '134' '94' '160' '120' '80' '' '191' '147' '103' '195' '150' '105'
 '111' '83' '55' '110' '13' '10' '7' '48' '36' '24']

 Fiber (g)
['0' '3' '2' '' '1' '4']

 Sugars (g)
['0' '2' '1' '36' '27' '18' '110' '83' '55' '167' '125' '107' '80' '53'
 '172' '132' '92' '176' '135' '94' '175' '134' '169' '130' '91' '173'
 '133' '93' '108' '81' '54' '112' '84' '56' '111' '159' '119' '151' '113'
 '75' '' '181' '139' '97' '185' '142' '99' '98' '109' '82']

 Protein (g)
['1' '0' '2' '10' '7' '5' '8' '6' '11' '9' '3' '4' '' '12' '16' '15' '14'
 '13' '19' '18']

 Weight Watcher Pnts
['4' '5' '126' '93.5' '63' '300' '219.5' '150' '570' '433' '285' '950.5'
 '710.5' '469.5' '567' '420' '283' '1304' '1001' '708' '998.5' '779' '561'
 '1051' '812' '582.5' '1291' '999' '707' '995.5' '767' '550' '1049' '810'
 '580.5' '899' '680' '450' '604' '448' '302' '657.5' '491.5' '1094' '833'
 '597' '942' '703' '476' '']
```

In [88]:
```
import numpy as np

dnkn2 = dnkn2.replace('', np.nan)
```

In [89]:
```
dnkn2.isnull().sum()
```

Out[89]:
```
Category               0
Item                   0
Calories               3
Total Fat (g)          3
Saturated Fat (g)      3
Trans Fat (g)          3
Cholesterol (mg)       3
Sodium (mg)            3
Carbs (g)              3
Fiber (g)              3
Sugars (g)             3
Protein (g)            3
Weight Watcher Pnts    3
dtype: int64
```

In [90]:
```
dnkn3 = dnkn2.dropna().copy()
```

In [91]:
```
dnkn3.shape
```

Out[91]:
```
(599, 13)
```

In [92]:
```
numeric_columns = [
    'Calories',
    'Total Fat (g)',
    'Saturated Fat (g)',
    'Trans Fat (g)',
    'Cholesterol (mg)',
    'Sodium (mg)',
    'Carbs (g)',
    'Fiber (g)',
    'Sugars (g)',
    'Protein (g)',
    'Weight Watcher Pnts']
```

In [94]:
```
for col in numeric_columns:
    dnkn3[col] = pd.to_numeric(dnkn3[col])
```

In [95]:
```
dnkn3.dtypes
```

Out[95]:
```
Category               object
Item                   object
Calories                int64
Total Fat (g)         float64
Saturated Fat (g)     float64
Trans Fat (g)         float64
Cholesterol (mg)        int64
Sodium (mg)             int64
Carbs (g)               int64
Fiber (g)               int64
Sugars (g)              int64
Protein (g)             int64
Weight Watcher Pnts   float64
dtype: object
```

In [96]:
```
print("========== MISSING VALUES ==========")
print(dnkn3.isnull().sum())

print("\n========== DATA TYPES ==========")
print(dnkn3.dtypes)

print("\n========== DUPLICATE RECORDS ==========")
print("Duplicates:", dnkn3.duplicated().sum())

print("\n========== CATEGORY CHECK ==========")
print(dnkn3['Category'].value_counts())

print("\n========== SPECIAL VALUE CHECK ==========")
print("Cholesterol Values:")
print(dnkn3['Cholesterol (mg)'].unique())

print("\nTotal Fat Values:")
print(dnkn3['Total Fat (g)'].unique())

print("\n========== DATASET SHAPE ==========")
print(dnkn3.shape)

print("\n========== SUMMARY STATISTICS ==========")
print(dnkn3.describe().round(2))
```

```
========== MISSING VALUES ==========
Category               0
Item                   0
Calories               0
Total Fat (g)          0
Saturated Fat (g)      0
Trans Fat (g)          0
Cholesterol (mg)       0
Sodium (mg)            0
Carbs (g)              0
Fiber (g)              0
Sugars (g)             0
Protein (g)            0
Weight Watcher Pnts    0
dtype: int64

========== DATA TYPES ==========
Category                object
Item                    object
Calories                 int64
Total Fat (g)          float64
Saturated Fat (g)      float64
Trans Fat (g)          float64
Cholesterol (mg)         int64
Sodium (mg)              int64
Carbs (g)                int64
Fiber (g)                int64
Sugars (g)               int64
Protein (g)              int64
Weight Watcher Pnts    float64
dtype: object

========== DUPLICATE RECORDS ==========
Duplicates: 0

========== CATEGORY CHECK ==========
Category
Hot Coffee              88
Iced Coffee             63
Hot Latte               63
Iced Latte              63
Frozen Coffee           47
Hot Macchiato           45
Iced Macchiato          42
Hot Cappuccino          42
Iced Cappuccino         42
Iced Tea                33
Hot Chocolate           28
Cold Brew Coffee        11
Coolatta                 9
Dunkin,Äô Refreshers     9
Soft Serve               8
Hot Americano            3
Iced Americano           3
Name: count, dtype: int64
```

```
========== SPECIAL VALUE CHECK ==========
Cholesterol Values:
[  0  35  25  20  15  10 130 100  70  40  30 105  80  55   5 125  50  45
  60]

Total Fat Values:
[ 0.  12.   9.   6.   4.5  3.  40.  31.  22.   5.  11.   8.  35.  26.
 17.  13.  14.  10.   7.   4.   1.   0.5 23.  21.  15.  16.   1.5  2.5
  2.  18.   3.5 20.  19.  27. ]

========== DATASET SHAPE ==========
(599, 13)

========== SUMMARY STATISTICS ==========
       Calories  Total Fat (g)  Saturated Fat (g)  Trans Fat (g)  \
count    599.00         599.00             599.00         599.00
mean     253.33           5.04               3.03           0.03
std      183.42           6.37               3.98           0.16
min        5.00           0.00               0.00           0.00
25%      130.00           0.00               0.00           0.00
50%      230.00           4.00               2.50           0.00
75%      340.00           8.00               4.50           0.00
max     1160.00          40.00              22.00           1.50

       Cholesterol (mg)  Sodium (mg)  Carbs (g)  Fiber (g)  Sugars (g)  \
count            599.00       599.00     599.00     599.00      599.00
mean              15.39       139.42      46.03       0.35       42.79
std               18.44       111.38      35.13       0.74       33.01
min                0.00         5.00       0.00       0.00        0.00
25%                0.00        60.00      25.00       0.00       23.50
50%               10.00       120.00      40.00       0.00       38.00
75%               25.00       190.00      63.00       0.00       57.00
max              130.00       750.00     195.00       4.00      185.00

       Protein (g)  Weight Watcher Pnts
count       599.00               599.00
mean          5.99               293.16
std           4.17               215.54
min           0.00                 4.00
25%           2.00               156.50
50%           6.00               266.00
75%           9.00               386.00
max          19.00              1352.00
```

In [97]:
```
dnkn3.to_csv('dnkn_clean.csv', index=False)
```

In [98]:
```
dnkn4 = pd.read_csv('dnkn_clean.csv')
```

In [99]:
```
dnkn4.head()
```

Out[99]:

	Category	Item	Calories	Total Fat (g)	Saturated Fat (g)	Trans Fat (g)	Cholesterol (mg)	Sodium (mg)	Carbs (g)
0	Cold Brew Coffee	Cold Brew Coffee, Large	5	0.0	0.0	0.0	0	20	0
1	Cold Brew Coffee	Cold Brew Coffee, Medium	5	0.0	0.0	0.0	0	15	0
2	Cold Brew Coffee	Cold Brew Coffee, Small	5	0.0	0.0	0.0	0	10	0
3	Cold Brew Coffee	Cold Brew Coffee with Cream, Large	120	12.0	6.0	0.0	35	65	2
4	Cold Brew Coffee	Cold Brew Coffee with Cream, Medium	90	9.0	4.5	0.0	25	45	1

For Deliverable Two, I focused on cleaning and preparing the Dunkin' Donuts nutritional dataset for analysis. I started to examine the data to identify missing values, duplicate records and inconsistencies. During this proccess, there was only one duplicate record. That record was the Ham and Cheese Rollups. I also discovered that the dataset contained food items that were not relevant to our beverage analysis, therefore I removed them to focus on the beverages. Additionally, I converted empty strings into NAN values and removed rows containing missing information. Moreover,I converted the nutritional columns into numerical data types so they could be used for calculations and comparisons. I also peformed several checks to make sure the dataset was properly cleaned. After completeing this proccess, the data set contained 599 rows and 13 columns. Overall, This process helped me understand how important data cleaning is because inaccurate or inconsistent information can affect the results of a business analysis.

In [ ]:

In [ ]: